

Functional Dependencies & Normalization

1. Why Normalization Exists — the Problem First

A poorly designed relation (one that groups together attributes that don't truly belong together) suffers from three concrete anomalies:

- **Insertion anomaly** — you can't insert a fact about one entity without also having data about an unrelated entity (e.g., can't add a new department's tax rate until at least one lot in that county exists).
- **Deletion anomaly** — deleting one fact accidentally destroys an unrelated fact (e.g., deleting the only employee in a department erases the fact that the department exists at all).
- **Update anomaly** — the same fact is stored redundantly in multiple rows, so updating it requires finding and changing *every* copy — miss one, and the data becomes inconsistent.

Normalization is the systematic process of decomposing relations to eliminate these anomalies, guided by a precise, formal tool: **functional dependencies**.

2. Functional Dependencies (FDs)

Definition: A functional dependency $X \rightarrow Y$ on relation schema R means: for any two tuples t_1, t_2 in *any* legal state of R , if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$. In plain terms: **the value of X uniquely determines the value of Y.**

Worked example — `EMP_PROJ(Ssn, Ename, Pnumber, Pname, Plocation, Hours)` :

<code>Ssn → Ename</code>	(an employee's SSN determines their name)
<code>Pnumber → {Pname, Plocation}</code>	(a project number determines its name and location)
<code>{Ssn, Pnumber} → Hours</code>	(an employee-project PAIR determines hours worked)

Critical properties of FDs

- **An FD is a property of the schema (the intended meaning of attributes), not of any one instance/table snapshot.** It must hold for **every** legal state of the relation, forever — not just the rows currently sitting in the table.
- **You cannot prove an FD true just by looking at data** — data can only ever **disprove** an FD (one counterexample is enough), never conclusively prove one holds in general, since the *next* row inserted could always violate it. FDs come from understanding the real-world **semantics** of the attributes, not from mining the current dataset.

Worked counterexample: in relation `TEACH(Teacher, Course, Text)`, if `Smith` teaches both `Data Structures` and `Data Management`, then `Teacher → Course` **does not hold** — this one pair of tuples is a direct counterexample.

3. Keys, Attribute Closure, and Candidate Keys via FDs

- **Attribute closure** X^+ — the set of **all** attributes functionally determined by X , given a set of FDs F (computed by repeatedly applying FDs: start with X , and keep adding any attribute A such that some FD $Y \rightarrow A$ exists with Y already inside the closure so far).
- X is a **superkey** of R if and only if X^+ includes **every** attribute of R .
- X is a **candidate key** if $X^+ = R$ **and** no proper subset of X also has this property (minimality) — this gives a formal, mechanical way to actually *find* candidate keys from a set of FDs, rather than guessing.

Armstrong's Axioms — inference rules for deriving new FDs

Given a set of FDs F , these rules let you derive **all** FDs logically implied by F (called F^+ , the closure of F):

Axiom	Rule	Meaning
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$	A set trivially determines any of its own subsets
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$	Adding the same extra attributes to both sides preserves the dependency
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	Dependencies chain together

Derived (secondary) rules, provable from the three above, but useful shortcuts:

| Rule | Statement |

|---|---|

| **Union** | If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$ |

| **Decomposition** | If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$ |

| **Pseudotransitivity** | If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$ |

Interview soundbite: "Armstrong's axioms are 'sound and complete' — sound means they never produce a false FD, complete means they can derive *every* FD that's actually logically implied. This is exactly what makes attribute closure computation a reliable, mechanical algorithm rather than a guessing game — you can always find every candidate key of a relation purely from its FDs, without needing to inspect actual data."

4. Normal Forms — 1NF through BCNF

Key principle: normalization proceeds through *progressively stricter* forms, each fixing a different, specific problem — a relation in a higher form automatically satisfies all lower forms.

First Normal Form (1NF)

Rule: every attribute must contain only **atomic (single, indivisible)** values — no repeating groups, no composite or multivalued attributes stored directly in one column.

- This is **baked into the relational model's very definition** (see the Relational Model topic) — 1NF isn't an optional design choice; it's a precondition for a table to even *be* a valid relation.
- **Fix:** if a table has a multivalued attribute (e.g., a `Dept` with multiple `Locations` crammed into one field), decompose it into a **separate relation** holding `(key, repeated-attribute)` pairs, one row per value.

Second Normal Form (2NF)

Rule: in 1NF, **and** every non-prime attribute is **fully** functionally dependent on **every** candidate key — i.e., no non-prime attribute depends on only *part* of a composite key (**no partial dependency**).

(A prime attribute is one that belongs to some candidate key; a non-prime attribute belongs to none.)

Worked example: `LOTS(Property_id#, County_name, Lot#, Area, Price, Tax_rate)`.

Candidate keys: `{Property_id#}` and `{County_name, Lot#}`.

```
FD1: Property_id# → County_name, Lot#, Area, Price, Tax_rate
FD2: {County_name, Lot#} → Property_id#, Area, Price, Tax_rate
FD3: County_name → Tax_rate           ← PROBLEM
FD4: Area → Price
```

FD3 violates 2NF: `Tax_rate` depends on `County_name` alone — only *part* of the composite candidate key `{County_name, Lot#}`, not the whole thing.

Decomposition:

```
LOTS1(Property_id#, County_name, Lot#, Area, Price)
LOTS2(County_name, Tax_rate)
```

Both are now in 2NF.

Third Normal Form (3NF)

Rule: in 2NF, **and** for every FD $X \rightarrow A$ in the relation, either (a) X is a superkey, **or** (b) A is a prime attribute (**no transitive dependency of a non-prime attribute on a key**).

Continuing the LOTS example: `LOTS1(Property_id#, County_name, Lot#, Area, Price)` still has

`FD4: Area  $\rightarrow$  Price`.

- `Area` is **not** a superkey.
- `Price` is **not** a prime attribute.
- $\rightarrow$ **FD4 violates 3NF** (Price depends transitively on the key, *through* Area, rather than directly).

Decomposition:

```
LOTS1A(Property_id#, County_name, Lot#, Area)
LOTS1B(Area, Price)
```

Both now in 3NF.

Why test for 2NF before 3NF, historically? 2NF and 3NF attack genuinely **different** problems (partial dependency vs. transitive dependency) — by definition, any relation in 3NF automatically already satisfies 2NF, but working through 2NF first, historically, made the *specific* problem being fixed at each stage clearer and easier to teach/diagnose.

Boyce-Codd Normal Form (BCNF)

Rule: for every nontrivial FD $X \rightarrow A$, X **must** be a superkey — **full stop**, with no exception for A being prime (that's exactly the "condition (b)" exception 3NF allows, which BCNF removes).

This makes BCNF strictly stronger than 3NF: every BCNF relation is in 3NF, but not every 3NF relation is in BCNF.

The classic case where 3NF $\neq$ BCNF — the TEACH example:

```
TEACH(Student, Course, Instructor)
```

Suppose: each instructor teaches only one course (`Instructor  $\rightarrow$  Course`), but a course can be taught by several instructors, and a student can take a course from any of them.

- `{Student, Course}` is a candidate key.
- `Instructor` is *also* effectively a determinant: `Instructor  $\rightarrow$  Course`.
- `Course` is a **prime** attribute (it's part of the candidate key `{Student, Course}`) — so this FD **satisfies 3NF's exception (b)**.
- But `Instructor` is **not a superkey** — so this FD **violates BCNF**, which allows no such exception.

Why this case matters: it demonstrates that 3NF can still tolerate a specific, subtle kind of redundancy (here: which course an instructor teaches gets repeated for every student they teach) that BCNF specifically catches and eliminates.

The BCNF decomposition trade-off — a genuinely important nuance: decomposing into BCNF can sometimes **lose functional dependency preservation** (see §5) — a dependency that held in the original relation can no longer be checked directly, because its attributes no longer coexist in the same decomposed relation. This is exactly why **BCNF is not always chosen over 3NF** in practice — 3NF guarantees dependency preservation is always achievable, BCNF does not.

Summary Table

Normal Form	Fixes	Rule
1NF	Non-atomic values	Every attribute holds only atomic values
2NF	Partial dependency	Every non-prime attribute fully depends on every candidate key
3NF	Transitive dependency	For every $X \rightarrow A$: X is a superkey, OR A is prime
BCNF	3NF's remaining exception	For every $X \rightarrow A$: X must be a superkey (no exception)

Interview soundbite: "1NF is about atomicity of individual values. 2NF is about a non-key attribute depending on only *part* of a composite key. 3NF is about a non-key attribute depending on another non-key attribute rather than the key directly. BCNF closes the one loophole 3NF still allows — a determinant that isn't a superkey, as long as what it determines happens to be part of some other candidate key."

5. Properties a Good Decomposition Must Have

When a relation is decomposed (into R_1, R_2) during normalization, two properties matter:

Property	Meaning	Is it mandatory?
Nonadditive (Lossless) Join	Joining the decomposed relations back together via natural join reproduces exactly the original relation — no spurious tuples, no lost tuples	Must always hold — non-negotiable
Dependency Preservation	Every original FD can still be checked using only attributes that coexist within <i>some</i> single decomposed relation, without needing to join relations back together just to verify a constraint	Desirable, but not always achievable (as seen in the BCNF/LOTS1A example above)

Simple test for lossless join on a binary decomposition (R decomposed into R_1, R_2): the decomposition is lossless if and only if $(R_1 \cap R_2)$ is a **key** of at least one of R_1 or R_2 .

Why lossless join is non-negotiable but dependency preservation isn't: losing the lossless-join property means the decomposition is flat-out **wrong** — joining the pieces back together can produce spurious tuples that never existed in the original data, silently corrupting query results. Losing dependency preservation is a genuine **inconvenience** (you'd need to join relations just to check a constraint that used to be checkable within one table) but doesn't corrupt the data itself — which is exactly why it's an accepted trade-off in some BCNF decompositions, but lossless join never is.

6. Multivalued Dependencies (MVD) & Fourth Normal Form (4NF)

Multivalued Dependency $x \twoheadrightarrow y$: for a fixed value of x , the set of associated y values is **completely independent** of the set of associated z values (where z is every other attribute) — both vary freely and combine in **every** combination.

The classic all-key problem — $EMP(Ename, Pname, Dname)$: an employee $Ename$ works on multiple projects ($Pname$) and has multiple dependents ($Dname$), and these two facts are **completely unrelated to each other** — yet naively storing them in one table forces **every combination** of that employee's projects $\times$ dependents to be listed, purely to preserve the "shape" of a table.

Ename	Pname	Dname
Smith	X	John
Smith	X	Anna
Smith	Y	John
Smith	Y	Anna

This has **no functional dependencies at all** (it's an "all-key" relation — the only key is all three attributes together) — so it's already technically in **BCNF**! But it's clearly redundant: **Pname** values and **Dname** values are being needlessly cross-multiplied. This is exactly the gap **4NF** exists to close.

Definition: R is in 4NF if, for every **nontrivial** MVD $X \twoheadrightarrow Y$, X is a **superkey**.

Decomposition: split into **EMP_PROJECTS(Ename, Pname)** and **EMP_DEPENDENTS(Ename, Dname)** — now each MVD is **trivial** within its own relation, and the redundant cross-multiplication disappears entirely.

Interview soundbite: "A relation can be in BCNF (no problematic *functional* dependencies) and still be badly redundant, because BCNF says nothing about *multivalued* dependencies. 4NF is specifically the fix for 'this table is forcing two independent one-to-many facts about the same entity to be cross-multiplied together' — an all-key relation with multiple independent multivalued attributes is the textbook symptom."

7. Join Dependency & Fifth Normal Form (5NF) — brief overview

Join Dependency $JD(R_1, R_2, \dots, R_n)$ — a relation R can be losslessly reconstructed by joining $R_1, R_2, \dots, R_n$ back together (a generalization of lossless-join decomposition to **more than two** pieces at once). An MVD is actually a special case of a JD where $n = 2$.

Definition: R is in 5NF (also called **Project-Join Normal Form, PJNF**) if, for every nontrivial join dependency, every R_i in the join dependency is a superkey of R .

- **5NF is rarely needed in practice** — the specific 3-or-more-way join dependencies it addresses are genuinely hard to detect and rare in real-world designs, unlike 3NF/BCNF issues, which come up constantly.
- **Real-world designs are typically normalized only up to 3NF or BCNF**, occasionally 4NF — 5NF is largely of theoretical/completeness interest for interviews (know it exists and roughly what it means, rather than needing deep fluency).

8. Denormalization — a deliberate step backward

Sometimes, after normalizing for correctness, designers **deliberately reintroduce some redundancy** (denormalize) for **performance** reasons — e.g., avoiding an expensive join on a very hot query path by duplicating a small amount of data.

This is a conscious, informed trade-off (correctness/storage efficiency vs. query performance) — not a mistake or a sign of bad design, as long as it's done deliberately and the resulting redundancy is carefully managed (e.g., via triggers or application logic keeping duplicated data in sync).

Interview Questions With Answers

Q1. Why can't a functional dependency be conclusively proven true just by inspecting the current data in a table?

Answer: A functional dependency is a constraint about the *meaning* of attributes that must hold across **every possible legal state** of the relation, not just the rows that happen to exist right now. The current data might simply not yet contain a counterexample — a future insert could violate the FD at any time. Data can definitively **disprove** an FD (one violating pair of tuples is sufficient), but it can never **prove** one holds in general, since that would require checking all possible future states, which is only knowable from understanding the real-world semantics of the attributes, not from the dataset itself.

Q2. What is attribute closure, and how does it let you find all candidate keys of a relation mechanically?

Answer: The closure of a set of attributes X (written X^+) is the set of all attributes functionally determined by X , computed by repeatedly applying the given FDs — starting with X and adding any attribute A for which some FD $Y \rightarrow A$ exists where Y is already known to be in the closure. X is a superkey exactly when X^+ includes every attribute of the relation, and X is a candidate key when $X^+ = R$ and no proper subset of X also achieves this (minimality) — this turns "find the candidate keys" from guesswork into a mechanical, repeatable computation directly from the given FDs.

Q3. State Armstrong's three axioms, and explain why they're described as "sound and complete."

Answer: Reflexivity: if Y is a subset of X , then $X \rightarrow Y$ always trivially holds. Augmentation: if $X \rightarrow Y$ holds, then adding the same extra attributes Z to both sides still holds: $XZ \rightarrow YZ$. Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$. They're "sound" because applying them to a true set of FDs can never produce a false FD, and "complete" because every FD that is genuinely logically implied by a given set of FDs can be derived using just these three rules — together, this guarantees that mechanically applying these axioms is a fully reliable way to compute the entire closure of a set of FDs, with nothing missed and nothing incorrectly added.

Q4. Using the LOTS example, explain precisely why FD3 ($\text{County_name} \rightarrow \text{Tax_rate}$) violates 2NF but a similar-looking dependency might not.

Answer: LOTS has candidate keys $\{\text{Property_id}\# \}$ and $\{\text{County_name}, \text{Lot}\# \}$. FD3 says Tax_rate depends on County_name alone — but County_name is only *part* of the composite candidate key

{County_name, Lot#} , not the whole key. This is precisely what 2NF forbids: a non-prime attribute (Tax_rate) must be fully dependent on an *entire* candidate key, not just part of one. If Tax_rate had instead depended on the full {County_name, Lot#} pair together (not derivable from County_name alone), it would not violate 2NF, since it would then depend on the *complete* key rather than a proper subset of it.

Q5. Why is BCNF described as "stronger" than 3NF, and what specific exception does 3NF allow that BCNF removes?

Answer: Both require every determinant of a nontrivial FD to be a superkey, *except* 3NF has an additional escape clause: it's also acceptable if the dependent attribute A is a *prime* attribute (part of some candidate key), even if the determinant X is not a superkey. BCNF removes this escape clause entirely — X must be a superkey, with no exception for A being prime. This makes BCNF strictly stronger (every BCNF relation is automatically in 3NF), but not every 3NF relation satisfies the stricter BCNF rule — exactly the gap the TEACH example demonstrates.

Q6. Walk through the TEACH example and explain exactly why it's in 3NF but not BCNF.

Answer: In TEACH(Student, Course, Instructor), assume each instructor teaches exactly one course (Instructor → Course) while a course can have multiple instructors, with {Student, Course} as the candidate key. The FD Instructor → Course has Course as its dependent attribute, and Course happens to be a *prime* attribute (part of the candidate key {Student, Course}) — this satisfies 3NF's exception (b), so the relation is in 3NF. But BCNF requires the determinant itself (Instructor) to be a superkey, and it isn't (knowing the instructor alone doesn't determine the student) — so this same FD violates BCNF, revealing redundancy (the fact "this instructor teaches this course" gets needlessly repeated once per student that instructor teaches).

Q7. Why can decomposing into BCNF sometimes lose dependency preservation, and why is this considered an acceptable trade-off while losing the lossless-join property never is?

Answer: BCNF decomposition can split a relation's attributes across pieces such that a dependency's determinant and dependent attribute no longer both exist together in any single decomposed relation — meaning that FD can no longer be verified just by checking one table; you'd need to join relations back together to check it, defeating some of the purpose of decomposing at all. This is accepted as a trade-off (rather than avoided) because it's merely an *inconvenience* — the constraint still logically holds, it's just harder to check directly. Losing the lossless-join property, in contrast, means joining the decomposed pieces back together can literally produce *incorrect, spurious data* that never existed in the original relation — a correctness failure, not just an inconvenience, which is exactly why lossless join is treated as absolutely mandatory while dependency preservation is only a "nice to have."

Q8. Why can a relation be in BCNF and still suffer from serious redundancy? What specific tool identifies and fixes this?

Answer: BCNF only constrains *functional* dependencies — it says nothing about *multivalued* dependencies, where an entity has two or more independent one-to-many facts (e.g., an employee's set of projects and their set of dependents) that get needlessly cross-multiplied together when stored in a single table with no actual functional dependencies at all (making it trivially satisfy BCNF, since an all-key relation has no FDs to violate). Fourth Normal Form (4NF) is the tool that specifically identifies this: it requires that for every nontrivial multivalued dependency $X \twoheadrightarrow Y$, X must be a superkey — catching exactly this "two independent multivalued facts forced into one table" pattern that BCNF is blind to.

Q9. Scenario: You're reviewing a schema `Book(ISBN, Title, AuthorName, GenreTag)` where a book can have multiple authors and multiple genre tags, and these two facts are completely independent of each other (an author list and a genre list for the same book don't influence one another). The current table has no functional dependencies beyond `ISBN → Title`, so it's technically in BCNF. Is this schema actually well-designed? If not, what would you do?

Answer: No — despite being in BCNF, this schema has exactly the multivalued dependency problem 4NF is designed to catch: `ISBN  $\twoheadrightarrow$  AuthorName` and `ISBN  $\twoheadrightarrow$  GenreTag` are two independent multivalued facts about the same book, and storing them together in one table forces every combination of that book's authors and genre tags to be listed as separate rows (e.g., a book with 2 authors and 3 genres would need 6 rows just to represent independent facts that should only need $2 + 3 = 5$ rows total). The fix is a 4NF-style decomposition: split into `Book_Authors(ISBN, AuthorName)` and `Book_Genres(ISBN, GenreTag)` (plus a `Book(ISBN, Title)` relation for the core facts) — each multivalued dependency becomes trivial within its own relation, eliminating the needless cross-multiplication entirely.